

Jeu de la Vie de Conway

Rapport Global - Itération 2

Projet long - TOB
ENSEEIH

12 mai 2025

Table des matières

1	Introduction	2
1.1	Contexte du projet	2
1.2	Méthodologie et organisation du travail	2
1.2.1	Utilisation de Git pour le versionnement	2
2	Améliorations de l'interface graphique	3
2.1	Interface graphique configurable	3
2.2	Interactions utilisateur	3
2.3	Mode Console Amélioré	3
3	Fonctionnalités avancées de simulation	5
3.1	Mode Pas à Pas	5
3.1.1	Implémentation technique	5
3.2	Contrôle de Vitesse	5
3.3	Mode Éducatif	5
4	Persistance et analyse des données	6
4.1	Système de sauvegarde et chargement	6
4.1.1	Format du fichier	6
4.1.2	Interface utilisateur	6
4.2	Statistiques en temps réel	6
5	Tests et qualité du code	7
5.1	Tests unitaires	7
5.2	Améliorations techniques	7
6	Conclusion et perspectives	8
6.1	Bilan de l'itération	8
6.2	Difficultés rencontrées	8
6.3	Perspectives d'évolution	8

Chapitre 1

Introduction

1.1 Contexte du projet

Ce document constitue le rapport global de la deuxième itération du projet "Jeu de la Vie de Conway" réalisé dans le cadre de notre formation universitaire. Cette itération s'est concentrée sur l'amélioration et l'extension de l'application développée précédemment, avec pour objectif d'enrichir l'expérience utilisateur et d'ajouter des fonctionnalités avancées.

Le Jeu de la Vie de Conway est un automate cellulaire imaginé par le mathématicien britannique John Conway en 1970. Il permet de simuler l'évolution de cellules selon des règles simples mais produisant des comportements complexes.

1.2 Méthodologie et organisation du travail

Pour cette itération, nous avons adopté une approche inspirée des méthodes agiles. L'équipe a procédé à un vote pondéré pour définir les priorités des fonctionnalités à implémenter. Cette approche nous a permis de :

- Identifier collectivement les fonctionnalités prioritaires
- Répartir le travail de manière équilibrée entre les membres
- Suivre régulièrement l'avancement global du projet

1.2.1 Utilisation de Git pour le versionnement

Un aspect essentiel de notre organisation a été l'utilisation intensive des branches Git pour structurer notre développement. Cette approche nous a permis de :

- Travailler en parallèle sur différentes fonctionnalités
- Isoler les développements de fonctionnalités dans des branches dédiées
- Assurer un suivi précis des modifications apportées
- Faciliter la revue de code et l'intégration des nouvelles fonctionnalités
- Maintenir un historique clair des évolutions du projet

À la fin de chaque développement majeur, une fusion (*merge*) était effectuée dans la branche principale après vérification du bon fonctionnement de la fonctionnalité.

Chapitre 2

Améliorations de l'interface graphique

2.1 Interface graphique configurable

L'interface graphique a été considérablement améliorée pour offrir une meilleure expérience utilisateur et une plus grande flexibilité :

- Dimensions de grille désormais personnalisables
- Taux de rafraîchissement ajustable via un slider
- Option de passage en plein écran
- Création d'une classe **StarterMenu** pour la configuration initiale
- Taille des cellules réduite de 20 à 10 pixels pour améliorer la visualisation

```
1 public JeuDeLaVieGUI(int HAUTEUR, int LARGEUR, int DELAI) {  
2     super("Jeu de la Vie");  
3     this.HAUTEUR = HAUTEUR;  
4     this.LARGEUR = LARGEUR;  
5     this.DELAI = DELAI;  
6     // ... reste du constructeur  
7 }
```

Listing 2.1 – Constructeur mis à jour de la classe JeuDeLaVieGUI

2.2 Interactions utilisateur

L'interface a été enrichie avec des fonctionnalités interactives pour une meilleure manipulation de la simulation :

- Possibilité de cliquer sur les cellules pour modifier leur état lorsque la simulation est en pause
- Boutons d'arrêt, démarrage et réinitialisation de la simulation
- Option pour activer ou désactiver la grille torique
- Option pour afficher ou masquer le quadrillage de la grille

2.3 Mode Console Amélioré

Le mode console a également été enrichi pour offrir plus de contrôle lors de l'exécution :

- Ajout d'un mode pas-à-pas avec l'argument **-step**
- Possibilité de basculer entre mode automatique et manuel

```
1 if (step) {  
2     Scanner sc = new Scanner(System.in);  
3     System.out.println("Appuyer sur entrer pour continuer...");  
4     sc.nextLine();  
5 }
```

Listing 2.2 – Extrait du mode console mis à jour

Chapitre 3

Fonctionnalités avancées de simulation

3.1 Mode Pas à Pas

Une fonctionnalité majeure ajoutée durant cette itération est le mode pas à pas, permettant de naviguer entre les générations :

- Navigation temporelle entre les générations (avancer/reculer)
- Boutons "Pas Avant" et "Pas Arrière" dans l'interface
- Visualisation claire des transitions d'une génération à l'autre

3.1.1 Implémentation technique

Pour permettre cette navigation temporelle, plusieurs mécanismes ont été mis en place :

- Création d'une liste historique des états de la grille (**List<Grille>**)
- Méthode **ajouterHistorique()** pour conserver les états successifs
- Méthode **copierGrille()** pour réaliser une copie profonde de l'état actuel
- Méthode **restaurerEtat(index)** pour revenir à un état antérieur
- Mécanisme de gestion des branches temporelles lors de modifications

3.2 Contrôle de Vitesse

Le contrôle de la vitesse de simulation a été implémenté pour permettre aux utilisateurs d'ajuster le rythme selon leurs besoins :

- Slider permettant d'ajuster la vitesse de simulation en temps réel
- Plage de vitesse allant de 50ms à 950ms entre chaque génération
- Réactivité du contrôle grâce à la reconfiguration dynamique du timer

3.3 Mode Éducatif

Une dimension pédagogique a été ajoutée à l'application via un mode éducatif dédié :

- Nouvelle classe **FenetreEducative** présentant une fenêtre modale
- Présentation du Jeu de la Vie et de ses règles d'évolution
- Guide d'utilisation de la simulation
- Présentation de l'intérêt scientifique et éducatif
- Catalogue des structures célèbres à explorer

La fenêtre éducatif est accessible à tout moment pendant l'utilisation de l'application via un bouton dédié dans l'interface principale.

Chapitre 4

Persistance et analyse des données

4.1 Système de sauvegarde et chargement

Un système complet de persistance des configurations a été développé :

- Sauvegarde des configurations dans des fichiers texte
- Chargement des configurations depuis des fichiers existants
- Format de fichier simple et efficace
- Gestion robuste des erreurs lors des opérations d'entrée/sortie

4.1.1 Format du fichier

Le format choisi pour les fichiers de sauvegarde est le suivant :

- Première ligne : hauteur,largeur,estTorique(true/false)
- Lignes suivantes : matrice de 0 et 1 séparés par des virgules

4.1.2 Interface utilisateur

L'interface a été enrichie avec :

- Boutons "Sauvegarder" et "Charger"
- Pause automatique temporaire lors des opérations sur les fichiers
- Messages de confirmation pour l'utilisateur
- Redémarrage automatique avec la nouvelle configuration si nécessaire

4.2 Statistiques en temps réel

Une nouvelle fonctionnalité d'analyse a été ajoutée pour suivre l'évolution de la simulation :

- Classe `StatistiquesJeuDeLaVie` dédiée à l'analyse de la grille
- Affichage du nombre de générations écoulées
- Comptage des cellules vivantes à la génération actuelle
- Suivi des naissances et des morts entre deux générations
- Calcul de la moyenne de cellules vivantes sur l'ensemble des générations

Ces statistiques sont affichées dans un panneau dédié en bas de l'interface graphique, offrant ainsi une vision synthétique de l'évolution du système en temps réel.

Chapitre 5

Tests et qualité du code

5.1 Tests unitaires

Des tests unitaires ont été mis en place pour garantir la fiabilité du code :

- Tests JUnit pour vérifier le bon fonctionnement de la classe `Cellule`
- Vérification du comportement des cellules (état vivant ou mort)
- Validation de l'évolution correcte des cellules selon les règles du jeu

5.2 Améliorations techniques

Plusieurs modifications structurelles ont été apportées pour améliorer la qualité du code :

- Séparation de la configuration et de la logique du jeu
- Meilleure gestion des états (plein écran, pause, etc.)
- Validation des entrées utilisateur pour éviter les erreurs
- Optimisation de la gestion mémoire, notamment pour l'historique des états

Chapitre 6

Conclusion et perspectives

6.1 Bilan de l'itération

Cette seconde itération a permis d'enrichir considérablement notre implémentation du Jeu de la Vie de Conway. Les principales améliorations apportées sont :

- Une interface graphique plus flexible et configurable
- Des contrôles avancés pour la navigation dans la simulation
- Des fonctionnalités éducatives pour comprendre les principes du jeu
- Un système de persistance pour sauvegarder et partager des configurations
- Des outils d'analyse statistique pour suivre l'évolution des générations

6.2 Difficultés rencontrées

Plusieurs défis techniques ont dû être relevés :

- Problèmes de gestion mémoire avec l'historique des états
- Réactivité du timer pour le contrôle de la vitesse
- Gestion des cas particuliers lors du chargement de grilles de tailles différentes

Ces difficultés ont été résolues par des ajustements techniques, notamment la suppression des états "futurs" lors de la création de nouvelles branches temporelles et l'arrêt/redémarrage dynamique du timer.

6.3 Perspectives d'évolution

Pour les prochaines itérations, plusieurs pistes d'amélioration sont envisagées :

- Bibliothèque de motifs prédéfinis (glisseurs, oscillateurs, etc.)
- Reconnaissance automatique des structures stables ou périodiques
- Export de l'évolution de la simulation en format vidéo
- Implémentation de variantes des règles du jeu
- Optimisation des performances pour les grilles de grande taille